Atomic-GDN2: TPU Kernels and Hierarchical Delta Routing for Hybrid Language Models

Akseleu Omirbay
Independent researcher

October 16, 2026

Abstract

This paper describes a TPU-oriented Gated DeltaNet-2 implementation in JAX/Pallas and a language-model experiment combining recurrent token mixing, Multi-head Latent Attention, and hierarchical delta routing. The kernel uses a three-stage forward path and a fused custom backward, with explicit chunk-level residuals and numerical constraints. The 307.13M-parameter hybrid model is compared with a 334.16M-parameter pure GDN-2 baseline in four fresh TPU v5e-8 runs, with two seeds per architecture and a common batch size of 32. At exactly 61,500 optimizer steps, after 4.030B training target tokens per run, hybrid achieves estimated validation BPB of 0.993838 ± 0.000111, compared with 1.005909 ± 0.000227 for the baseline; the difference is -0.012071 BPB. The uncertainty shown is sample standard deviation over two seeds. Hybrid has lower BPB at every shared logged evaluation in both seeds and reaches the evaluated BPB thresholds using 14.3–18.5% fewer tokens and 10.4–13.6% less logged notebook time. These results compare complete architectures: they do not isolate the contribution of delta routing, equalize FLOPs, or establish downstream generalization. Historical kernel timings are reported separately from these model results, with reproducibility caveats. Code, exact run evidence, analysis data and checkpoint export metadata accompany the paper.

1 Introduction

Recurrent linear-attention models provide a useful alternative to a growing attention cache, but an efficient recurrence also needs an implementation suited to the accelerator. Gated DeltaNet-2 separates channel-wise erasing and writing in a matrix-state update [1]. On a TPU, the implementation must organize that update around chunked matrix operations, on-chip working storage and explicit forward/backward interfaces. The mathematical model, the kernel implementation and the surrounding language model are distinct levels of the system; their evidence should be kept distinct as well.

This work studies two connected engineering questions. First, how can a GDN-2 kernel be structured in JAX/Pallas for TPU execution while making numerical assumptions explicit? Second, how does a recurrent/attention hybrid with hierarchical depth routing compare with a pure GDN-2 model under a practical, fixed session budget? The hybrid replaces one recurrent mixer per block with Multi-head Latent Attention (MLA) [2] and adapts the scoring and additive routing equations of Delta Attention Residuals (DAR) [3] to previous-block summaries and earlier within-block deltas.

The contributions are: (i) a documented kernel decomposition with a fused backward and selective recomputation; (ii) an explicit implementation of hierarchical routing in a GDN-2/MLA language model; and (iii) a two-seed, four-run comparison with complete logged learning curves, first-observed threshold crossings and immutable source provenance. The two architectures share the same kernel implementation, so their quality difference is not a comparison of Pallas against a scan backend.

The experimental result is intentionally narrow. At a shared observed token count, hybrid has lower validation loss despite having fewer parameters. This is evidence about these two architectures on this corpus and schedule, not a state-of-the-art claim or an isolated DAR ablation. The article also separates historical kernel timing reports from current model results rather than interpreting a large microbenchmark ratio as model-level speedup.

2 Background and mathematical specification

2.1 Gated DeltaNet-2 recurrence

Let $S_t \in \mathbb{R}^{d_k \times d_v}$ be the recurrent state. The reference implemented in the accompanying kernel repository uses channel-wise erase and write vectors b_t, w_t, a decay vector α_t, and query/key/value vectors q_t, k_t, v_t:

$$\bar{S}_t = \mathrm{Diag}(\alpha_t) S_{t-1}, \tag{1}$$

$$u_t = b_t \odot k_t, \qquad \widehat{v}_t = w_t \odot v_t - \bar{S}_t^T u_t, \tag{2}$$

$$S_t = \bar{S}_t + k_t \widehat{v}_t^T, \qquad o_t = \gamma S_t^T q_t. \tag{3}$$

Here $\gamma = 1/\sqrt{d_k}$ in the language model. This explicitly distinguishes the erase and write channels; a scalar-gate formula without w_t would not specify this API. The implementation applies $b_t = \sigma(W_b x_t)$ and

$$g_t = -\min\left(\mathrm{softplus}(W_g x_t + c_g), \frac{160}{b_{s2}}\right), \qquad \alpha_t = \exp(g_t). \tag{4}$$

For the training configuration $b_{s2} = 32$, $g_t \in [-5, 0]$. This bound belongs to the chosen balanced two-leg (BTL) decay computation, not to every implementation of GDN-2. The deployed chunked computation also includes damping and sanitization, described below; its agreement with an undamped mathematical recurrence must be evaluated under matched settings.

2.2 Delta routing along depth

DAR routes changes introduced by earlier sublayers rather than only cumulative hidden states [3]. We retain its learned query-vector scoring, normalized keys, unprojected values and additive retrieval, and adapt the source grouping to the hybrid blocks. For a source set $\mathcal{V}_i$ available before sublayer i,

$$a_{ij} = \mathrm{softmax}_{j \in \mathcal{V}_i}\left(z_i^T \,\mathrm{RMSNorm}_{\mathrm{key}}(v_j)\right), \tag{5}$$

$$r_i = \sum_{j \in \mathcal{V}_i} a_{ij} v_j, \qquad \delta_i = F_i(\mathrm{Norm}_{\mathrm{pre}}(x_i + r_i)), \tag{6}$$

$$x_{i+1} = \mathrm{sanitize}(x_i + \delta_i) \quad \text{for token mixers.} \tag{7}$$

The query z_i is a learned vector, initialized at zero, rather than a projection of x_i. A zero query produces uniform weights over a nonempty source set; it does not make retrieval zero. If there are no sources, retrieval is zero. The ordinary residual stream remains present. The FFN has an analogous routed input with pre-LayerNorm and an ordinary additive residual update; it is not clipped at the same update site as the mixers.

3 The `atomic_gdn2` kernel

3.1 Execution structure and memory

The `atomic-ops` distribution contains two distinct kernel generations: the older `atomic_ops` port and the TPU-oriented `atomic_gdn2` implementation used here. The current model imports the latter through a small cached binding. Its forward pass has three Pallas stages: decay-aware score construction, a leaf-batched WY solve, and fused WY-factor reconstruction plus an inter-chunk scan. With chunk length 128, a length-2048 segment has 16 recurrence iterations over chunks.

The custom VJP passes saved chunk-level quantities to a reverse scan in `backward_mega`. A single Pallas call per segment combines the stages labelled B2, B1, B3, B4 and B5 in the code, returning gradients for q, k, v, w, b, g and the initial state. Long sequences are segmented; the default maximum is 32 chunks per segment, and recurrent state is passed across segment boundaries. Thus "one fused backward" means one call per segment, not one launch for every possible sequence length or for all of forward/backward.

Memory is reduced through structured residual reuse and selective recomputation. The forward retains, among other quantities, score/solve matrices, chunk-start states and updated values. The backward reconstructs local factors such as w_p, k_g, q_g. These saved arrays scale with the number of chunks. Neither constant-memory training nor the absence of all HBM intermediates is claimed. The recurrent carry is maintained inside the chunk loop, but that should not be confused with keeping the entire training computation in on-chip storage.

3.2 Precision, damping and numerical envelope

The current configuration uses different matmul modes for different stages. Score and CD operations use `highest`; the ladder solve and B1/B2/B3 operations use `bf16x3`; B4 explicitly selects `highest`. FP32 tensor storage and a matmul precision mode are different properties. The bf16x3 helper splits each operand into high and low components and forms three products:

$$AB \approx A_{hi}B_{hi} + A_{hi}B_{lo} + A_{lo}B_{hi}. \tag{8}$$

It does not split each operand into three stored bf16 components.

The WY solve implemented by the shipped code is

$$A = (I + (1 - \varepsilon)A_{kk})^{-1}, \qquad \varepsilon = 10^{-3}. \tag{9}$$

This rescales the triangular coupling; it is not diagonal loading by εI. The BTL score factorization balances two exponential legs. The configuration ties the maximum per-token log decay to the inner block size so the bound $g_{max}b_{s2}/2 = 80$ stays below the nominal exponential limit 88. Kernel-level clipping uses 10^4; the hybrid has separate mixer/residual sanitization at 10^3. The frequency and quality effect of these safeguards were not measured.

3.3 Historical timing evidence and its scope

Table 1 reproduces the values stored in the kernel repository's historical `attestation/scaling.json`, not a benchmark newly executed for this paper revision. They concern a single TPU v5e chip, $B = 4$, head dimension 128, inner block size 64, and the supplied associative-scan wrapper. The largest reported ratio is 467.64×. The model runs use eight devices, eight heads and inner block size 32, so the ratio is not transferred to their throughput.

Table 1: Historical reported kernel forward-plus-backward timings. The wrappers require the reproduction qualifications in the text; these are not new measurements of this release.

Heads	Length	Scan wrapper (ms)	Pallas (ms)	Reported ratio
2	2048	288.796	1.498	192.83×
2	4096	575.320	2.781	206.90×
4	2048	744.459	2.485	299.59×
4	4096	1501.515	4.668	321.69×
6	2048	1198.748	2.563	467.64×
6	4096	2398.292	5.352	448.10×

A static audit of the current reproduction notebook found duplicate argument passing and nonidentical initial-state, output-scale and final-state handling in the old scan wrapper. The supplied loss includes the final-state norm, so these differences matter. The old composition also normalizes transition matrices. A corrected reproduction harness now gates timing on matched forward and gradient comparisons, but has not been timed on TPU. Consequently, the historical ratios are descriptive reports, not newly established equivalent-workload speedups.

Table 2 records the historical forward-output comparisons against a JAX reference. They concern two shapes and do not constitute a general backward bound. The larger aggregate contains 145 PASS entries and one OLD-baseline MQAR OOM; its entries mix numerical, training and status checks. Raw per-notebook logs were not committed with that aggregate. A separate 200.17× end-to-end figure in older documentation lacks its matching raw artifact in the inspected aggregate and is not used as a result here. The complete evidence notes and exact files are linked in the repository.

4 Hybrid architecture

The model has seven blocks, each with three GDN-2 mixers followed by one causal MLA mixer and one SwiGLU FFN. MLA uses a rank-512 KV representation and separate 64-dimensional non-positional and rotary query/key components per head. The model configuration is given in Table 3. In comparison, each baseline block has four GDN-2 mixers and one FFN. The baseline has 334,156,800 parameters and hybrid

Table 2: Historically reported forward relative L_2 error, Pallas versus the JAX reference, at length 2048. These are source-attestation values, not rerun results.

Heads	Relative L_2
4	2.486×10^{-6}
6	2.481×10^{-6}

has 307,131,392, approximately 8.09% fewer. These are measured current parameter counts; the older 342M Q/K-router model is a different implementation.

Table 3: Current model configuration. A token mixer is counted separately from the one FFN per block. Both models use the same external kernel implementation.

Setting	Baseline	hybrid
Parameters	334,156,800	307,131,392
Token mixers / blocks	28 / 7	28 / 7
GDN-2 / MLA mixers	28 / 0	21 / 7
SwiGLU FFNs	7	7
Hidden / FFN dimension	1024 / 3584	1024 / 3584
Heads / head dimension	8 / 128	8 / 128
MLA KV rank	—	512
MLA non-RoPE / RoPE dimensions	—	64 / 64
DAR query	—	1024-vector per router
Vocabulary / context	50,257 / 2048	50,257 / 2048
Dropout	0.1	0.1
Tied embeddings	Yes	Yes
Model compute / parameters	bf16 / fp32	bf16 / fp32

Within block b, the source set before mixer i is

$$\mathcal{V}_{b,i} = \{s_1, \ldots, s_{b-1}\} \cup \{\delta_{b,1}, \ldots, \delta_{b,i-1}\}. \tag{10}$$

The FFN receives the same previous-block summaries plus all four earlier mixer deltas. After the FFN, the block appends $s_b = x_{out} - x_{in}$, including the FFN change. Only earlier sublayers can contribute; no layer reads its own newly computed delta. These are depth sources at the same sequence position, not persistent memory across independent sequences. The current design does not contain the previous standalone IntraBlockAttention Q/K projection mechanism.

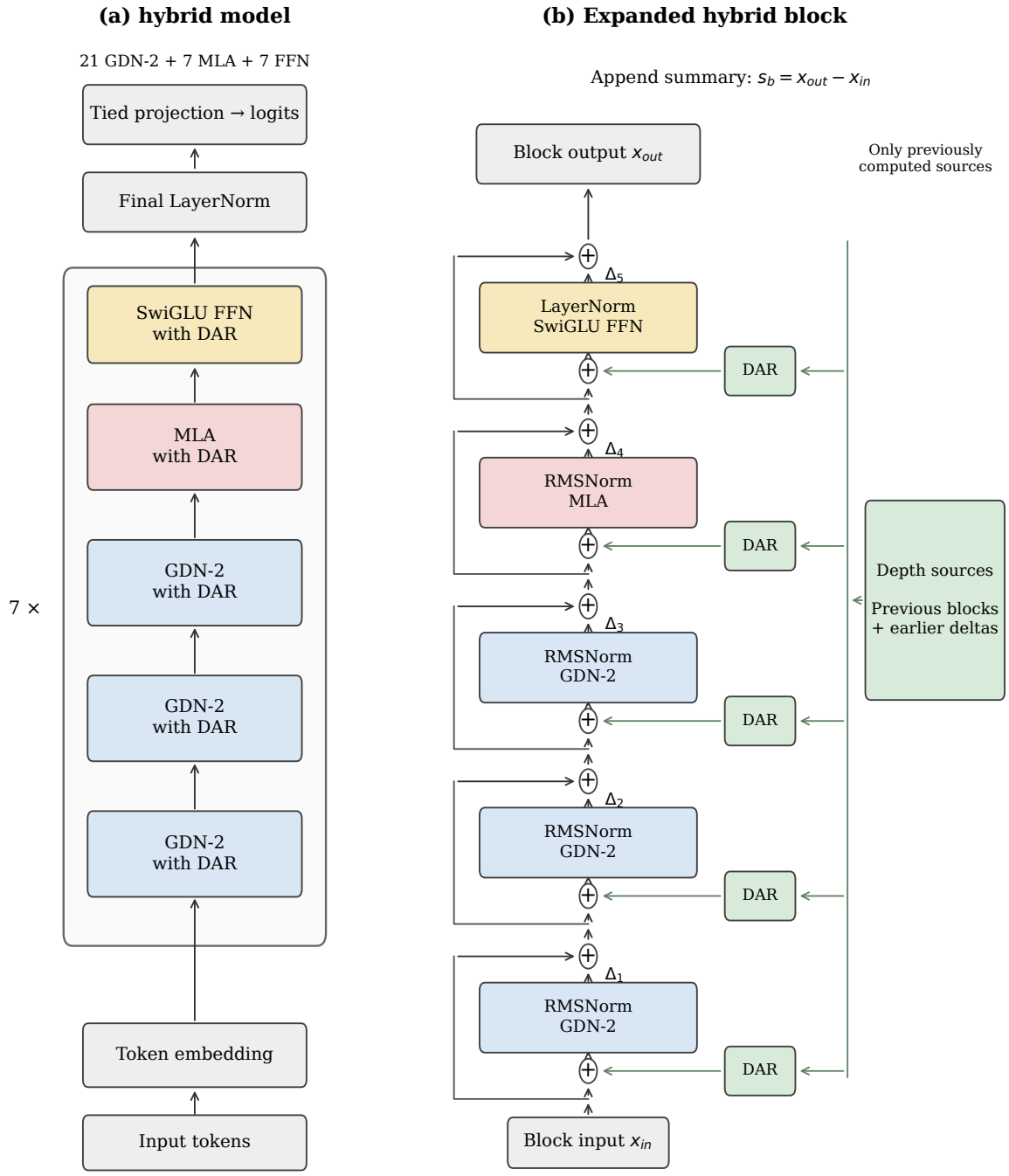


Figure 1: The current hybrid computation, with a diagram layout inspired by the stacked residual presentation of Vaswani et al. [5]. (a) Seven blocks, each containing three GDN-2 mixers, one MLA mixer and one FFN. (b) Sequential sublayers with an ordinary residual bypass and an additive DAR input before normalization. Mixer inputs use RMSNorm; the FFN uses LayerNorm. The depth-source box means summaries and deltas available before the indicated stage; it is not a simultaneous all-layer attention operation. Sanitization locations are specified in the implementation.

5 Experiments

5.1 Data, optimization and run protocol

All four runs were trained from scratch on the uint16 GPT-2-tokenized `windmaple/openwebtext-gpt2` corpus. The available files contain 9,035,582,489 training tokens and 4,434,606 validation tokens. The loader forms nonoverlapping windows of 2049 tokens, uses 2048 labels per window, and drops incomplete batches. Final evaluation covers 67 full batches, or 4,390,912 target tokens. The configured intermediate limit of 100 batches exceeds the available 67, so intermediate and final evaluations use the same available windows. There is no independent held-out test result.

Reported BPB is an estimate from the average token cross-entropy ℓ:

$$\mathrm{BPB}_{\mathrm{est}} = \frac{\ell}{\ln(2)\, 4.2}. \tag{11}$$

The constant 4.2 bytes per GPT-2 token is not measured on each evaluated sequence; therefore the metric is labelled estimated BPB rather than exact byte-level BPB.

Both architectures use batch 32, context 2048 and gradient accumulation 1. Initialization seeds are 1 and 2; the paired data seeds are 43 and 44, and dropout seeds are 124 and 125. The optimizer uses Muon [6] for eligible two-dimensional matrices excluding embeddings, and AdamW for the remaining parameters. Peak learning rate is 3×10^{-4}, weight decay 0.1 and global gradient clipping 1.0. Muon uses momentum 0.95 and five Newton–Schulz iterations; AdamW uses $(\beta_1, \beta_2) = (0.9, 0.95)$.

Each isolated notebook has a 32,040-second (8h54m) limit including setup and compilation, with a 600-second finalization reserve inside the limit. Learning rate follows an elapsed-time warmup/cosine rule with a first-100-update warmup bound and a 0.1 floor ratio. Equal optimizer steps therefore do not imply identical learning-rate trajectories. Each run uses a fresh optimizer and a shuffled single pass over the available train data, ending when its time budget expires. None completed the entire data pass. Validation is logged every 1500 steps; checkpoints are saved every 5000 steps and at finalization.

Table 4: Actual completed runs, before analytical curve truncation. Logged notebook time is the last console timestamp and includes setup/finalization; it is not a billing measurement.

Model	Seed	Final step	Train tokens (B)	Notebook h	Status
Baseline	1	65,000	4.259840	8.768	Complete
Baseline	2	65,000	4.259840	8.743	Complete
hybrid	1	62,673	4.107338	8.808	Complete
hybrid	2	62,683	4.107993	8.801	Complete

5.2 Validation learning curves and matched-step comparison

Figure 2A shows the mean and sample standard deviation over the two seeds. Hybrid is lower at every one of the 41 shared logged evaluations, from step 1500 to 61,500, in both seeds. Selected values appear in Table 5; the complete per-seed table is in the accompanying CSV and Appendix B.

At exactly 61,500 steps, both models have processed 4,030,464,000 training target tokens. Baseline reaches 1.005909 ± 0.000227 estimated BPB and hybrid reaches 0.993838 ± 0.000111. With Δ = hybrid − Baseline, the difference is -0.012071 BPB. The same kernel implementation is used by both architectures. This result does not separate MLA, routing, normalization or parameter-count effects.

For a near-step endpoint presentation, baseline is truncated at 63,000 and hybrid uses its observed final steps 62,673 and 62,683. Their respective means are 1.004928 and 0.993311; $\Delta = -0.011617$. This analytical cutoff was selected after the runs and is not an exactly token-matched endpoint. The original baseline evaluations through 65,000 remain in the released logs; Table 4 reports actual completion rather than the cutoff. There is no fabricated hybrid observation at 63,000 or exact evaluation at 1.5B tokens.

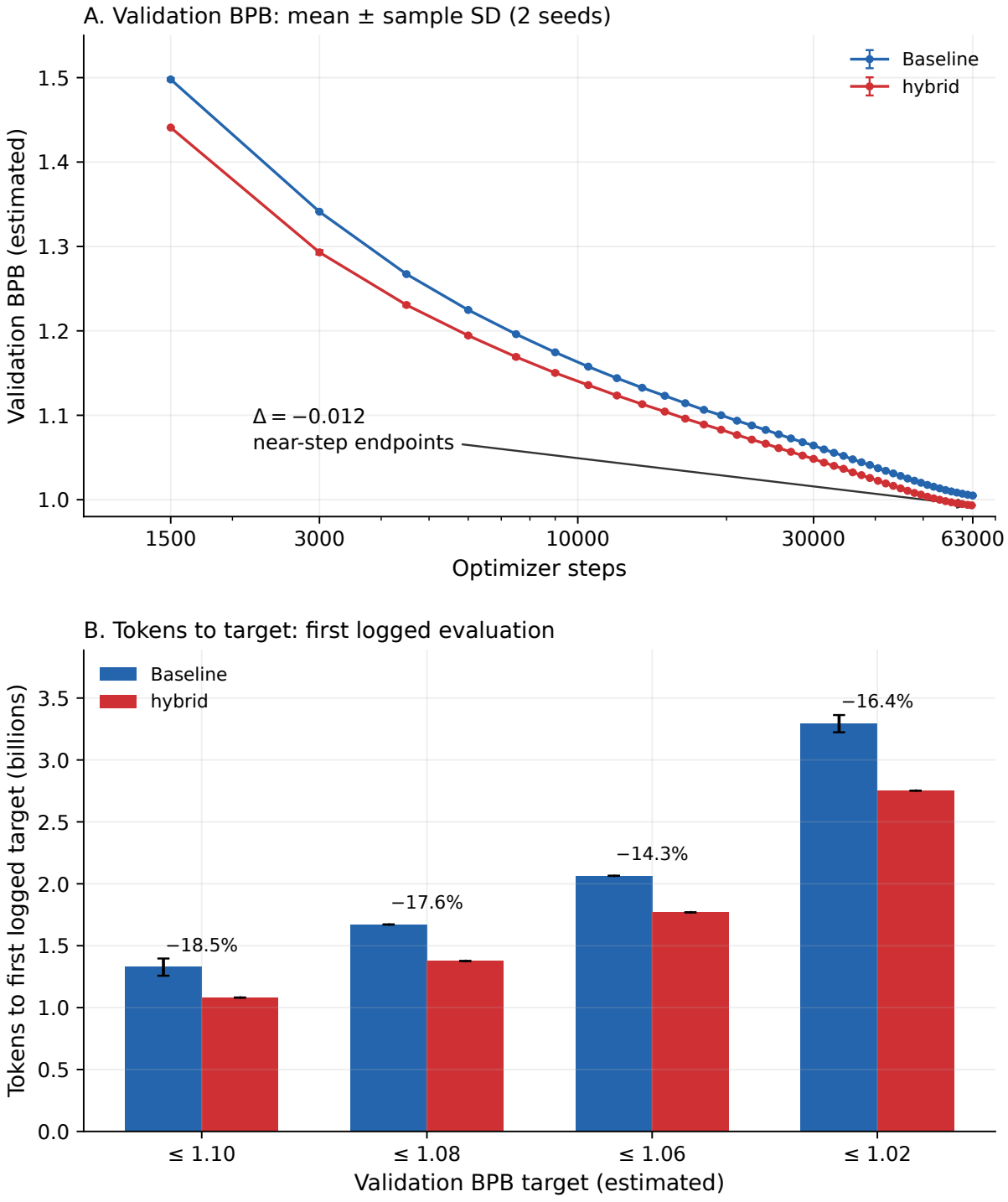


Figure 2: Validation learning curves and token requirements. (A) Means and sample SD over two seeds; the endpoint annotation uses Baseline 63,000 and hybrid 62,673/62,683 and is not an exact token match. (B) Training target tokens at the first logged evaluation meeting each threshold, aggregated over seeds. No crossing interpolation is used. Estimated BPB divides token CE by $\ln(2) \times 4.2$.

Table 5: Selected shared-step estimated BPB results, mean ± sample SD across two seeds. The final row is a separately labelled near-step comparison, not exact token matching.

Step	Tokens (B)	Baseline	hybrid	Δ
1,500	0.098	1.497840 ± 0.001396	1.440855 ± 0.000309	-0.056985
15,000	0.983	1.123090 ± 0.000266	1.104386 ± 0.000481	-0.018704
22,500	1.475	1.087951 ± 0.000609	1.071201 ± 0.000241	-0.016750
24,000	1.573	1.082673 ± 0.000202	1.066318 ± 0.000001	-0.016355
30,000	1.966	1.064235 ± 0.000363	1.048363 ± 0.000689	-0.015872
45,000	2.949	1.028161 ± 0.000065	1.013422 ± 0.000016	-0.014738
60,000	3.932	1.007089 ± 0.000269	0.994866 ± 0.000159	-0.012224
61,500	4.030	1.005909 ± 0.000227	0.993838 ± 0.000111	-0.012071
Near-step*	—	1.004928 ± 0.000254	0.993311 ± 0.000176	−0.011617

*Baseline 63,000; hybrid 62,673/62,683.

The advantage is largest at the earliest recorded evaluation: at step 1500 the mean difference is −0.056985 BPB. It narrows to −0.012071 at the last common step. Figure 3 displays the paired differences directly. This observed trend does not establish a shared asymptotic loss floor, an early-warmup mechanism, or the outcome of continuing training. A descriptive log-step slope is supplied with the plot, but adjacent evaluations are correlated and it is not a significance test.

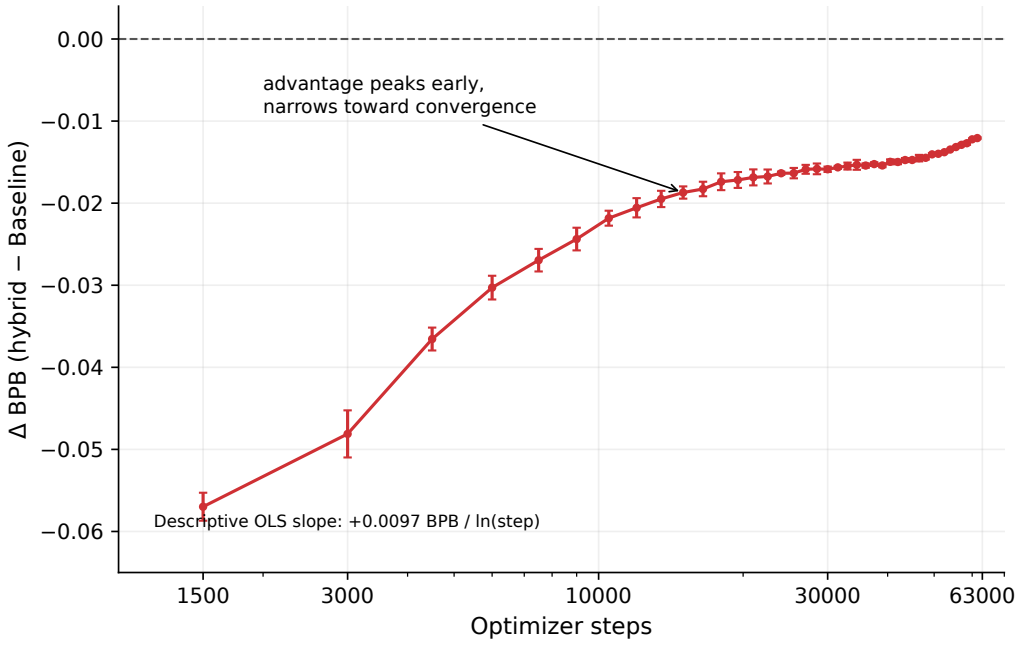


Figure 3: Paired estimated BPB differences at identical logged steps. Negative values favor hybrid. Error bars show sample SD of the two seed-paired differences, not a confidence interval. The curve stops at the last common observation, 61,500 steps.

5.3 Token and time to target

For each seed separately, the threshold crossing is the first recorded validation at or below the target. We then average those step/token/time values across seeds; this is not a crossing computed from the mean BPB curve. Table 6 gives the four evaluated thresholds. Token savings are 14.29–18.52%, smaller than the superseded single-seed historical claims. A zero SD in observed crossing steps means both seeds crossed on the same 1500-step grid point; it does not locate the true crossing exactly.

Token efficiency and elapsed-time efficiency are different measurements. The latter includes installation, compilation, validation and checkpoint overhead up to the crossing. Periodic saves sometimes waited roughly 200–243 seconds in the logs. Consequently, the wall-time result reflects the observed execution protocol rather

Table 6: First-observed target crossings. Steps are mean ± sample SD, $n = 2$. Notebook time is measured from the beginning of the notebook console, excluding the TPU queue.

Target BPB	Baseline steps	hybrid steps	Fewer tokens	Less time
≤ 1.10	20250 ± 1061	16500 ± 0	18.52%	13.48%
≤ 1.08	25500 ± 0	21000 ± 0	17.65%	13.55%
≤ 1.06	31500 ± 0	27000 ± 0	14.29%	10.38%
≤ 1.02	50250 ± 1061	42000 ± 0	16.42%	12.51%

than a pure kernel or steady-state throughput benchmark. The wall-time bars and exact per-seed crossing records are included in the appendix and machine-readable tables.

5.4 Run integrity, source provenance and checkpoint export

All four notebooks report completion and contain a nonempty SUCCESS record. Recorded updates have finite loss/gradient norms and zero skipped steps; each runtime preflight reports eight TPU devices and finite forward/backward execution. Hybrid's routing branch has a nonzero preflight task gradient. These checks establish observed execution health, not numerical equivalence to a reference at every training step.

The baseline and hybrid training source commits are respectively `bf80e315` and `697a10d3`; the common kernel commit is `0dbc59cf`. Each run retains its source-file hashes, resolved configuration, dataset file sizes and edge hashes, runtime preflight, metrics and console log. A publication snapshot is not misrepresented as the original training commit. Model parameter exports retain the original Flax tensor paths and orientations in safetensors; optimizer state is not part of inference weights. Export manifests record counts, shapes, finite checks and hashes. The documented loader is required; these files are not advertised as drop-in Transformers models.

6 Related work

Gated DeltaNet-2 supplies the channel-wise erase/write recurrence and its chunkwise formulation [1]; MLA supplies the compressed key/value attention component [2]. The delta-routing scoring and additive equations follow DAR [3], while the source grouping here combines previous block changes and local earlier deltas in a GDN-2/MLA backbone. We do not claim to have introduced those underlying mechanisms. The block composition and engineering integration are the object of this experiment.

Pallas exposes accelerator-specific kernel programming within JAX [4]. The kernel repository distinguishes its older Triton-derived port from the current TPU-oriented implementation. Comparisons with an associative-scan wrapper are useful for implementation analysis, but do not rank this work against all optimized linear attention or state-space kernels. The diagram adopts the clear stacked-residual visual language of Transformer diagrams [5], not their encoder–decoder topology or post-normalization ordering. Muon is used as an optimizer component rather than introduced or independently benchmarked here [6].

7 Limitations

Replication and evaluation scope. Two seeds expose the observed run-to-run variation but give a weak estimate of its distribution. Sample SD is not a confidence interval. The study uses one corpus, one tokenizer and context length 2048, with repeated validation on the available split. There is no independent final test, downstream benchmark or systematic generation study. The consistent sign in both seeds does not establish a universal advantage.

Attribution and resource matching. The architectures differ in MLA, routing and normalization details, parameter counts and FLOPs. No component ablation isolates DAR. Equal target tokens and batch size do not imply equal compute. Both use the same kernel; a historical Pallas/scan ratio cannot explain the architecture comparison as a backend change. Longer-context behavior and comparisons with tuned external implementations remain unmeasured.

Schedule and stopping. Learning rates depend on elapsed time, so models at the same step can have different learning-rate histories. The time-limited runs did not finish the available data pass. The selected near-step cutoff is post hoc and differs slightly between architectures and hybrid seeds; the exact common-step comparison and complete raw logs are therefore reported alongside it. No optimizer-reset artifact from the superseded experiment is attributed to these fresh runs.

Metric and crossing resolution. BPB uses a fixed bytes-per-token conversion. Validation drops the final incomplete batch. Threshold estimates are limited by the evaluation grid, and observed time includes overhead. Neither standard deviations from two seeds nor the descriptive trend fit justify a formal significance or asymptotic convergence claim.

Numerics, artifacts and reproducibility. The implementation uses clipping, damping and NaN replacement. Their activation rates and effect on quality are not separately characterized. Historical kernel attestations use older source/content identifiers and incomplete raw provenance; the previous reproducer had workload-equivalence defects. Its corrected parity-gated replacement is not a newly measured result. Finite training and successful checkpoint publication do not prove backward equivalence or independent reproduction. The release distinguishes lossless weight serialization checks from a full rerun of evaluation.

8 Conclusion

Atomic-GDN2 combines a documented TPU kernel implementation with an explicit hierarchical delta-routing hybrid and a four-run training comparison. At exactly 4.030B training target tokens, hybrid improves estimated validation BPB by 0.012071 over the pure GDN-2 baseline, with lower BPB at all shared evaluations in both seeds and approximately 8.09% fewer parameters. The observed savings to the evaluated quality thresholds are 14.3–18.5% in tokens and 10.4–13.6% in logged notebook time. These are bounded architectural results, not an isolated explanation of DAR's effect. The next informative experiments are additional seeds, component ablations, a fixed-token learning-rate schedule and independent task evaluation. A parity-gated kernel rerun is also needed before treating the historical largest speedup as a current equivalent-workload claim.

Code and artifact availability. The publication repository is `https://github.com/Akseleu-J/atomic-gdn2`. Kernel implementation and detailed historical evidence are at `https://github.com/Akseleu-J/atomic-ops`; checkpoint exports and model card are at `https://huggingface.co/Akseleu/Atomic-GDN2`. The model repository contains the source of this paper, vector figures, exact tables and a ZIP of all four run records. The old model repository is retained separately as a private archive. Readers should use the immutable training commits and per-file manifests rather than assume current main is the training snapshot. The availability statements describe the accompanying release, not an arXiv acceptance or submission.

References

[1] A. Hatamizadeh, Y. Choi and J. Kautz. *Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention*. arXiv:2605.22791, 2026. `https://arxiv.org/abs/2605.22791`.

[2] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv:2405.04434, 2024.

[3] C. Luo, Z. Cai and J. Hu. *Delta Attention Residuals*. arXiv:2605.18855, 2026. `https://arxiv.org/abs/2605.18855`.

[4] JAX authors. *Pallas: a JAX kernel language*. `https://docs.jax.dev/en/latest/pallas/`.

[5] A. Vaswani et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017. arXiv:1706.03762.

[6] K. Jordan. *Muon: An optimizer for hidden layers in neural networks*, 2024. `https://github.com/KellerJordan/Muon`.

[7] A. Omirbay. *atomic-ops: GDN-2 kernels and attestation*. `https://github.com/Akseleu-J/atomic-ops`.

A Per-seed and wall-time views

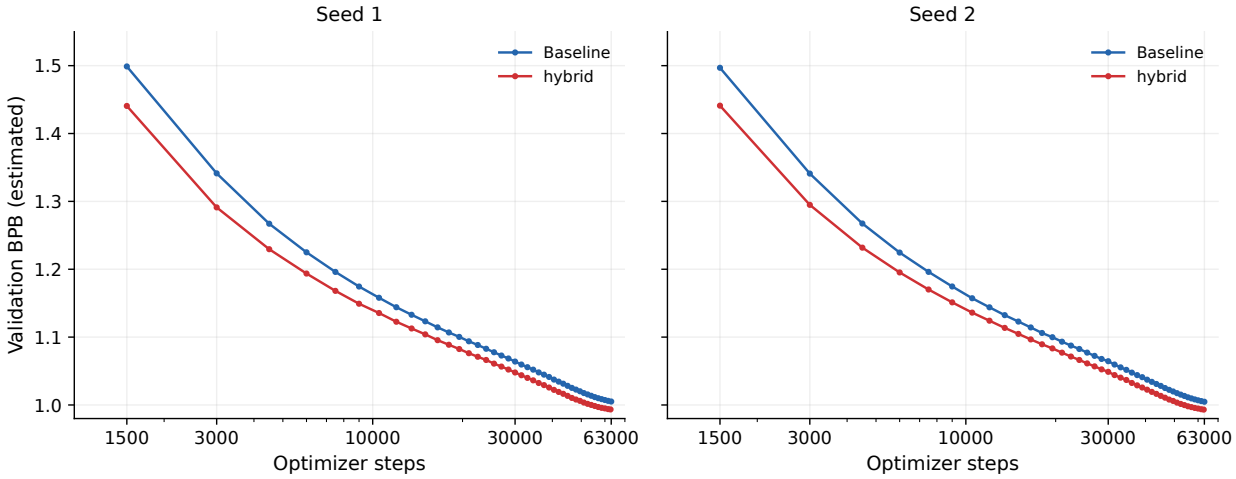


Figure 4: The two seed pairs shown separately, with the same axes and analytical cutoffs. Hybrid has lower estimated BPB at every common recorded step in each pair.

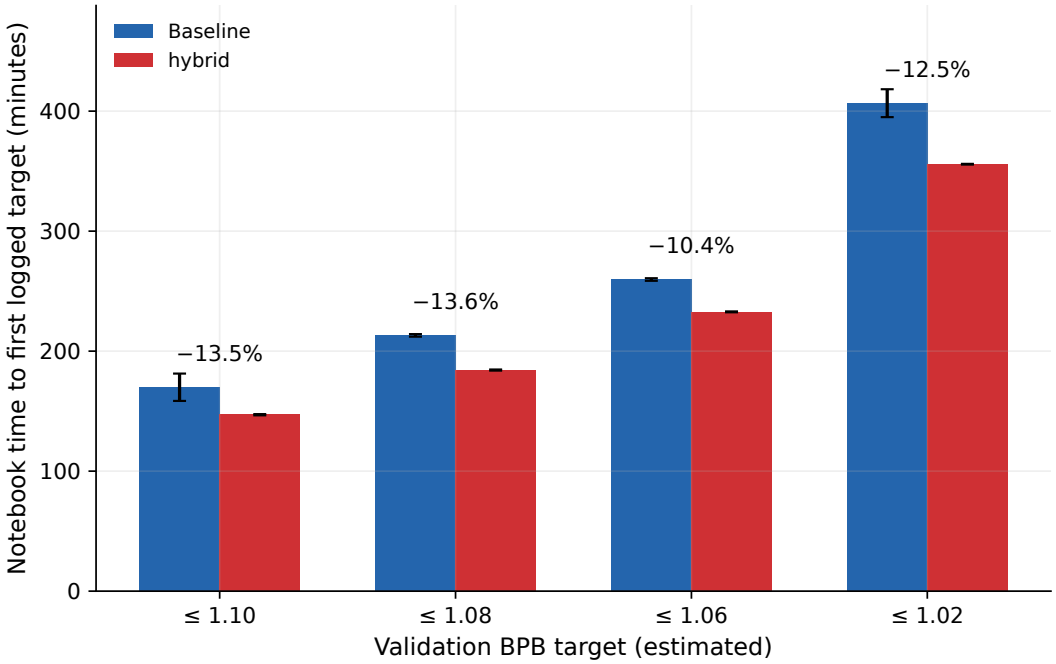


Figure 5: Notebook time at the first logged target crossing, including setup/compilation and earlier validation/checkpoint overhead, excluding queue time. Bars are means and error bars are sample SD over two seeds.

B Complete shared-step results

Table 7: All exact shared-step observations. Means and sample SD, $n = 2$ seeds.

Step	Baseline mean	Baseline SD	hybrid mean	hybrid SD
1,500	1.497840	0.001396	1.440855	0.000309
3,000	1.341130	0.000234	1.293017	0.002635
4,500	1.267232	0.000218	1.230678	0.001611
6,000	1.224777	0.000286	1.194487	0.001159

Step	Baseline mean	Baseline SD	hybrid mean	hybrid SD
7,500	1.196091	0.000044	1.169143	0.001419
9,000	1.174573	0.000057	1.150200	0.001438
10,500	1.157625	0.000497	1.135794	0.000414
12,000	1.143967	0.000096	1.123404	0.001077
13,500	1.132642	0.000383	1.113156	0.000608
15,000	1.123090	0.000266	1.104386	0.000481
16,500	1.114313	0.000076	1.096039	0.000808
18,000	1.106475	0.000540	1.089082	0.000482
19,500	1.100083	0.000319	1.082908	0.000649
21,000	1.093547	0.000421	1.076687	0.000554
22,500	1.087951	0.000609	1.071201	0.000241
24,000	1.082673	0.000202	1.066318	0.000001
25,500	1.077439	0.000387	1.061097	0.000246
27,000	1.072607	0.000334	1.056729	0.000208
28,500	1.068146	0.000483	1.052323	0.000171
30,000	1.064235	0.000363	1.048363	0.000689
31,500	1.059580	0.000013	1.043941	0.000164
33,000	1.055560	0.000162	1.040078	0.000262
34,500	1.051903	0.000203	1.036575	0.000404
36,000	1.047839	0.000228	1.032420	0.000039
37,500	1.044347	0.000479	1.029106	0.000254
39,000	1.041129	0.000207	1.025723	0.000013
40,500	1.037354	0.000106	1.022389	0.000195
42,000	1.034230	0.000228	1.019247	0.000043
43,500	1.031153	0.000299	1.016406	0.000054
45,000	1.028161	0.000065	1.013422	0.000016
46,500	1.025070	0.000140	1.010554	0.000257
48,000	1.022480	0.000246	1.008005	0.000037
49,500	1.020054	0.000211	1.006000	0.000064
51,000	1.017499	0.000085	1.003513	0.000065
52,500	1.015390	0.000206	1.001598	0.000097
54,000	1.013420	0.000184	0.999953	0.000177
55,500	1.011431	0.000231	0.998266	0.000297
57,000	1.009829	0.000212	0.996935	0.000077
58,500	1.008393	0.000291	0.995696	0.000099
60,000	1.007089	0.000269	0.994866	0.000159
61,500	1.005909	0.000227	0.993838	0.000111

The near-step endpoints are deliberately outside the exact shared-step table. Their per-seed values are Baseline 1.005107974/1.004748581 at step 63,000 and hybrid 0.993435231/0.993186593 at steps 62,673/62,683. Original baseline final values at 65,000 are 1.004100976 and 1.003824738. All these observations remain in the raw records.

C Reproduction records

The machine-readable attestation contains source commits and hashes, the full evolution, per-seed threshold crossings and the exact/near-step alignment distinction. The artifact ZIP retains the original metrics JSONL, notebook stdout/stderr and training log for each run, plus config, runtime checks and checkpoint metadata. Source/dataset edge hashes are not substitutes for full independent reproduction or a full-corpus checksum.

Weight exports preserve FP32 values and Flax tensor naming. Each export manifest specifies the source step, seed, tensor count, element count and digest. The default weight paths identify a concrete seed rather than an averaged or merged model. The model card explains the difference between current and legacy weights, intended research use and the absence of instruction tuning. Optimizer state is retained in the original training checkpoint format, not silently embedded into an inference safetensors file.